

```

"""
=====

Twitter / X Sentiment Analysis
ShadowFox Data Science Internship – Intermediate Task
Tool: VADER (Valence Aware Dictionary and sEntiment Reasoner)
=====

SETUP (run once in your terminal):
    pip install vaderSentiment pandas matplotlib seaborn

TO USE YOUR OWN DATASET:
    Replace the `sample_tweets` list below with:
        df = pd.read_csv("your_file.csv")
        df = df.rename(columns={"text": "tweet"})    # match your column name
"""

# — 1. IMPORTS —————
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import re
import warnings
warnings.filterwarnings("ignore")

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

sns.set_theme(style="whitegrid")
plt.rcParams["figure.dpi"] = 120

# — 2. DATASET —————
sample_tweets = [
    # Positive
    "Just got promoted! Best day of my life! 🎉 #blessed #happy",
    "The weather is absolutely gorgeous today. Love it! ☀️",
    "This new album is AMAZING! Every single track is a banger 🔥",
    "Feeling so grateful for my amazing friends and family ❤️",
    "Just finished my first 5K run! So proud of myself 💪",
    "Had the most wonderful dinner with old friends tonight 😊",
    "Python is such a beautiful language, loving this data science journey!",
    "Incredible sunrise this morning. Nature never fails to amaze me 🌄",
    "Finally got my dream job offer!! Screaming with joy right now 🥳",
    "This movie was absolutely breathtaking. 10/10 would recommend!",
    "Volunteered at the food bank today. Feeling warm and fulfilled 🧡",

```

"My team won the championship! What an unbelievable performance! 🏆",
 "The kindness of strangers restored my faith in humanity today.",
 "New coffee shop in town is perfect. Great vibes and amazing latte ☕",
 "Just adopted a puppy! My heart is so full right now 🐶",
 # Negative
 "This is the worst service I've ever experienced. Absolutely furious! 😡",
 "I can't believe they cancelled the show. So disappointed and sad 😞",
 "Traffic is horrible today. Been stuck for 2 hours. Hate this commute.",
 "Failed my exam again. Feeling like a complete failure right now.",
 "The government's new policy is a disaster. Terrible decision.",
 "Lost my wallet with all my cards. This day couldn't get any worse.",
 "So tired of all the negativity and hate on this platform. 😫",
 "Customer service was rude and unhelpful. Never shopping there again!",
 "This headache won't go away. Feeling miserable all day.",
 "Another data breach? I'm so sick of companies not protecting our data.",
 "My flight got cancelled with no refund. Absolutely unacceptable! 😡",
 "Disappointed with the lack of transparency from the authorities.",
 "Feeling so lonely and burnt out. Nothing seems to be going right.",
 "That new restaurant was overpriced and the food was disgusting. 🤢",
 "Can't sleep again. Anxiety is just overwhelming tonight.",
 # Neutral
 "The meeting has been rescheduled to 3pm tomorrow.",
 "Just read an interesting article about climate change statistics.",
 "The new software update is available for download.",
 "Watching the news. A lot happening in the world today.",
 "The train arrives at platform 4 at 6:30pm.",
 "Reminder: quarterly report is due by end of this week.",
 "Scientists have discovered a new species of deep-sea fish.",
 "The library will be closed on public holidays.",
 "According to the survey, 60% of users prefer dark mode.",
 "New regulations for electric vehicles come into effect next month.",
 "The conference will be held virtually this year.",
 "Today's high temperature is expected to be around 24 degrees Celsius.",
 "The quarterly GDP figures were released this morning.",
 "A new study examines the effects of screen time on sleep.",
 "The bus route 42 will be diverted due to road works.",

]

```
df = pd.DataFrame({"tweet": sample_tweets})
print(f"✅ Dataset loaded: {len(df)} tweets\n")
```

```
# — 3. TEXT CLEANING —————
def clean_tweet(text):
    """Clean tweet for display (keep original for VADER scoring)."""
    text = re.sub(r"http\S+|www\S+", "", text)
    text = re.sub(r"@w+", "", text)
```

```

    text = re.sub(r"#(\w+)", r"\1", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

df["cleaned_tweet"] = df["tweet"].apply(clean_tweet)

# — 4. VADER SCORING —————
analyzer = SentimentIntensityAnalyzer()

def get_scores(text):
    return analyzer.polarity_scores(text)

def classify(compound):
    if compound >= 0.05:
        return "Positive"
    elif compound <= -0.05:
        return "Negative"
    else:
        return "Neutral"

scores = df["tweet"].apply(get_scores)
df["neg"] = scores.apply(lambda x: x["neg"])
df["neu"] = scores.apply(lambda x: x["neu"])
df["pos"] = scores.apply(lambda x: x["pos"])
df["compound"] = scores.apply(lambda x: x["compound"])
df["sentiment"] = df["compound"].apply(classify)

print("Sentiment Counts:")
print(df["sentiment"].value_counts().to_string())
print()

# — 5. VISUALISATIONS —————
palette = {"Positive": "#22C55E", "Neutral": "#94A3B8", "Negative": "#EF4444"}
counts = df["sentiment"].value_counts()
order = ["Positive", "Neutral", "Negative"]

fig, axes = plt.subplots(2, 3, figsize=(18, 11))
fig.suptitle("Twitter / X Sentiment Analysis – VADER",
             fontsize=18, fontweight="bold", y=0.98)

# 1. Pie Chart
ax = axes[0, 0]
colors_pie = [palette[s] for s in counts.index]
wedges, texts, autotexts = ax.pie(
    counts, labels=counts.index, autopct="%1.1f%%",

```

```

        colors=colors_pie, startangle=140,
        wedgeprops={"edgecolor": "white", "linewidth": 2.5})
for at in autotexts:
    at.set_fontsize(11); at.set_fontweight("bold"); at.set_color("white")
ax.set_title("Sentiment Distribution", fontsize=13, fontweight="bold")

# 2. Bar Chart – Counts
ax = axes[0, 1]
bars = ax.bar(counts.index, counts.values,
              color=[palette[s] for s in counts.index],
              edgecolor="white", linewidth=1.5, width=0.5)
for bar, val in zip(bars, counts.values):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.2,
            str(val), ha="center", va="bottom", fontsize=12, fontweight="bold")
ax.set_title("Tweet Count per Class", fontsize=13, fontweight="bold")
ax.set_ylabel("Count")
ax.set_ylim(0, counts.max() + 3)
sns.despine(ax=ax)

# 3. Histogram – Compound Score
ax = axes[0, 2]
ax.hist(df["compound"], bins=20, color="#6366F1",
        edgecolor="white", linewidth=1.2, alpha=0.85)
ax.axvline(0.05, color="#22C55E", linestyle="--", lw=1.8, label="+0.05")
ax.axvline(-0.05, color="#EF4444", linestyle="--", lw=1.8, label="-0.05")
ax.axvline(df["compound"].mean(), color="#F59E0B", lw=2,
           label=f"Mean={df['compound'].mean():.2f}")
ax.set_title("Compound Score Distribution", fontsize=13, fontweight="bold")
ax.set_xlabel("Compound Score"); ax.set_ylabel("Frequency")
ax.legend(fontsize=9)
sns.despine(ax=ax)

# 4. Box Plot
ax = axes[1, 0]
sns.boxplot(data=df, x="sentiment", y="compound", order=order,
            palette=palette, width=0.45, ax=ax, linewidth=1.5)
sns.stripplot(data=df, x="sentiment", y="compound", order=order,
              palette=palette, alpha=0.5, jitter=True, size=5, ax=ax)
ax.axhline(0, color="grey", linestyle=":", lw=1.2)
ax.set_title("Compound Score by Class", fontsize=13, fontweight="bold")
ax.set_xlabel("Sentiment"); ax.set_ylabel("Compound Score")
sns.despine(ax=ax)

# 5. Stacked Bar – score components
ax = axes[1, 1]
score_means = df.groupby("sentiment")[["pos", "neu", "neg"]].mean().loc[order]
score_means.plot(kind="bar", stacked=True, ax=ax,

```

```

        color=["#22C55E", "#94A3B8", "#EF4444"],
        edgecolor="white", linewidth=1)
ax.set_title("Avg Score Components by Class", fontsize=13, fontweight="bold")
ax.set_xlabel("Sentiment"); ax.set_ylabel("Average Score")
ax.set_xticklabels(order, rotation=0)
ax.legend(["pos", "neu", "neg"], loc="upper right", fontsize=9)
sns.despine(ax=ax)

# 6. KDE - compound score density
ax = axes[1, 2]
for sent in order:
    subset = df[df["sentiment"] == sent]["compound"]
    subset.plot.kde(ax=ax, color=palette[sent], linewidth=2.5, label=sent)
ax.axvline(0, color="grey", linestyle=":", lw=1.2)
ax.set_title("Compound Score Density (KDE)", fontsize=13, fontweight="bold")
ax.set_xlabel("Compound Score"); ax.set_ylabel("Density")
ax.legend(fontsize=10)
sns.despine(ax=ax)

plt.tight_layout()
plt.savefig("sentiment_analysis_dashboard.png", bbox_inches="tight", dpi=150)
plt.show()
print("✅ Dashboard saved as sentiment_analysis_dashboard.png")

# — 6. SUMMARY REPORT —————
print("\n" + "="*55)
print("      SENTIMENT ANALYSIS - SUMMARY REPORT")
print("="*55)
total = len(df)
for s in order:
    cnt = (df["sentiment"] == s).sum()
    pct = cnt / total * 100
    avg = df[df["sentiment"] == s]["compound"].mean()
    print(f"  {s:<10}: {cnt:>3} tweets ({pct:5.1f}%) Avg compound: {avg:+.3f}")
print("-"*55)
print(f" Overall mean compound : {df['compound'].mean():+.4f}")
print(f" Overall std compound : {df['compound'].std():.4f}")
print(f" Most positive score   : {df['compound'].max():+.4f}")
print(f" Most negative score   : {df['compound'].min():+.4f}")
print("-"*55)
print("\n🟢 Most Positive Tweet:")
print(" ", df.loc[df["compound"].idxmax(), "tweet"])
print("\n🔴 Most Negative Tweet:")
print(" ", df.loc[df["compound"].idxmin(), "tweet"])
print("\n✅ Analysis complete!")

```

